



On utilise un ABR pour classer des villes selon différents critères.

le fichier [villes.csv](#) contient la liste des 200 plus grandes villes de France dans un ordre aléatoire.

Chaque ligne de ce fichier constitue un **enregistrement** relatif à une ville et contient les 5 informations suivantes :

- ✚ Nom de la ville.
- ✚ Numéro du département.
- ✚ Nombre d'habitants.
- ✚ Superficie (en km²)
- ✚ Rang au niveau national (critère : nombre d'habitants)

Ex 1 : Les deux premières lignes de ce fichier :

Saint-Priest,69,40944,29.7,155
Versailles,78,85761,26.2,46

1. Créer la classe Ville (dans un fichier nommé **TD_Villes_classe_ville**) avec les attributs d'instances suivants :

```
class Ville :
    """classe définissant une ville de France.
    Chaque ville a 4 attributs :
    - nom
    - numero de departement
    - population (hab)
    - superficie (km^2)
    - rang au niveau national (place)"""
```

```
def __init__(self,liste) :
    self.nom = liste[0]
    self.departement = int(liste[1])
    self.population = int(liste[2])
    self.superficie = float(liste[3])
    self.rang = int(liste[4])
```

création d'un objet:

```
ligne_liste = ['Nice', '6', '343123', '71.9', '5'] #enregistrement sous forme de liste
ville=Ville(ligne_liste)
```

- a. Vérifier l'accès aux différents attributs.
- b. Écrire une méthode **get_rang(self)**, qui retourne le rang de la ville.
- c. Écrire une méthode **get_superficie(self)**, qui retourne la superficie de la ville.
- d. Écrire une méthode **__str__(self)**, qui retourne une chaîne donnant toutes les informations à disposition sur la ville (avec leurs unités).

Par exemple :

« Nice (06), qui compte 343123 habitants et couvre un territoire de 71.9 km², est la 5ème ville de France. »

- e. Écrire une méthode **affiche_nom(self)**, qui affiche le nom de la ville.

2. On utilise une classe analogue à la classe Noeud à laquelle on ajoute une méthode **__str__()** :

- a. Éditer dans un fichier nommé **TD_Villes_ABR_ville_rang** (qui importera le fichier **TD_Villes_classe_ville**) le code suivant :

```
class Abr_ville :
```

Le constructeur

```
def __init__(self, v, g=None, d=None):
    """prend en paramètre
    v l'instance de la classe Ville correspondant à une ligne de ville extraite du fichier,
    g un arbre gauche et
    d un arbre droit"""

    self._ville = v
    self._gauche = g
    self._droit = d

def __str__(self):
    return str(self._ville.affiche_nom())
```



Chaque valeur de nœud est donc un objet instance de la classe Ville.

- b. Rajouter la méthode **insérer(self,v)** qui insère l'enregistrement liste d'une ville dans l'ABR des villes, **ordonné suivant le rang de la ville** (si une ville **v** est racine, tous les nœuds figurant dans le sous-arbre gauche ont un rang strictement inférieur à celui de **v**, et inversement à gauche).
3. Création de l'ABR :
 - a. Créer en l'extrayant du fichier csv, une liste **liste_villes** contenant les lignes du fichier villes.csv sous forme de listes.



Rappel pour extraire une table d'un fichier csv :

```
import csv
liste_villes=[]
with open("villes.csv",'r',encoding='utf-8') as fichier:
    lecteur=csv.reader(fichier,delimiter=',')
    for ligne in lecteur:
        liste_villes.append(ligne)
```

- b. Construire un arbre équilibré (gauche droite) à partir de **liste_villes**
4. Simulation et Exploitation de l'arbre :
 - a. Récupérer la méthode (ou la fonction) **parcours_infixe()** et l'adapter pour lui faire afficher les villes par ordre croissant de leur rang.
 - b. On souhaite maintenant faire afficher les villes par ordre croissant de leur superficie. Modifier le code en conséquence. (On peut créer un ABR générique doté d'un attribut de classe de la classe **ABR_Ville** qui définit l'ordre régissant la construction de l'ABR)
 - c. En utilisant cette nouvelle construction, écrire une méthode **recherche_max(self)** qui renvoie la ville ayant la plus grande superficie.
 - d. Écrire une méthode **rechercher(self,rg)**, qui retourne la ville dont le rang vaut **rg**